

{ Python }

Interview Prep Guide

100 Questions with Concepts, Examples & Coding Problems

10

Basic

15

Concepts

15

Intermediate

10

Advanced

50

Coding

Covers Python 3.x · Beginner to Advanced · Interview Ready

Table of Contents

Basic (10 questions)

- | | | | |
|------|--|------|---|
| Q001 | What is Python? | Q006 | What is a dictionary in Python? |
| Q002 | What are Python's key features? | Q007 | What are Python sets? |
| Q003 | What is PEP 8? | Q008 | How do you swap two variables in Python? |
| Q004 | What are Python's built-in data types? | Q009 | What is the difference between '==' and 'is'? |
| Q005 | What is the difference between a list and a tuple? | Q010 | What are Python keywords? |

Concepts (15 questions)

- | | | | |
|------|---|------|--|
| Q011 | What is a Python decorator? | Q019 | What is a closure? |
| Q012 | What is a Python generator? | Q020 | What is monkey patching? |
| Q013 | What is a list comprehension? | Q021 | What is the difference between __str__ and __repr__? |
| Q014 | What are *args and **kwargs? | Q022 | What is method resolution order (MRO)? |
| Q015 | What is a lambda function? | Q023 | What are Python's dunder (magic) methods? |
| Q016 | What is the difference between shallow copy and d...? | Q024 | What is a context manager? |
| Q017 | What is Python's GIL? | Q025 | What is the difference between staticmethod and c... |
| Q018 | What are Python's scope rules (LEGB)? | | |

Intermediate (15 questions)

- | | | | |
|------|--|------|--|
| Q026 | How does Python manage memory? | Q034 | What is pickling in Python? |
| Q027 | What is the difference between mutable and immuta... | Q035 | What is the difference between pass, continue, an... |
| Q028 | What are iterators and iterables? | Q036 | What are Python modules and packages? |
| Q029 | What is the difference between map, filter, and r... | Q037 | What is __init__.py? |
| Q030 | What is exception handling in Python? | Q038 | What is the 'with' statement? |
| Q031 | How do you create a custom exception? | Q039 | What is string formatting in Python? |
| Q032 | What is multiple inheritance? Explain the diamond... | Q040 | What are Python's built-in functions for sequences? |
| Q033 | What are Python's file I/O modes? | | |

Advanced (10 questions)

- | | | | |
|------|---------------------------------|------|--|
| Q041 | What is async/await in Python? | Q046 | What is the difference between threading and mult... |
| Q042 | What are metaclasses in Python? | Q047 | What are __slots__ in Python? |
| Q043 | What are descriptors? | Q048 | What is functools.lru_cache? |
| Q044 | Explain Python's dataclasses. | Q049 | What are Python type hints? |
| Q045 | What is Python's abc module? | Q050 | What is memoisation and how do you implement it? |

Coding Problems (50 questions)

- | | | | |
|------|--|------|--|
| Q051 | Reverse a string without using slicing. | Q076 | Rotate a list by k positions. |
| Q052 | Check if a string is a palindrome. | Q077 | Find the missing number in 1..n. |
| Q053 | Find the factorial of a number. | Q078 | Implement a Caesar cipher. |
| Q054 | Print the Fibonacci sequence up to n terms. | Q079 | Write a function to transpose a matrix. |
| Q055 | Find duplicate elements in a list. | Q080 | Implement a function that returns the power set. |
| Q056 | Count the occurrences of each element in a list. | Q081 | Count words in a string. |
| Q057 | Flatten a nested list. | Q082 | Find the longest common prefix in a list of strin... |
| Q058 | Implement binary search. | Q083 | Implement quick sort. |
| Q059 | Implement bubble sort. | Q084 | Find all prime factors of a number. |
| Q060 | Find two numbers that sum to a target. | Q085 | Convert decimal to binary without bin(). |
| Q061 | Check if two strings are anagrams. | Q086 | Implement a basic expression calculator (+/-). |

- Q062 Find the most frequent element in a list.
- Q063 Merge two sorted lists.
- Q064 Implement a stack using a list.
- Q065 Implement a queue using collections.deque.
- Q066 Count vowels in a string.
- Q067 Find largest and smallest in a list without built...
- Q068 Remove duplicates from a list preserving order.
- Q069 Implement a function to check for balanced bracke...
- Q070 Generate all permutations of a string.
- Q071 Implement merge sort.
- Q072 Find common elements between two lists.
- Q073 Implement a simple linked list.
- Q074 Check if a number is prime.
- Q075 Calculate the GCD of two numbers.
- Q087 Find the intersection of two sorted arrays.
- Q088 Implement strlen without len().
- Q089 Sum of digits of a number.
- Q090 Check if a number is an Armstrong (narcissistic) ...
- Q091 Generate Pascal's triangle.
- Q092 Find the longest substring without repeating char...
- Q093 Implement an LRU cache from scratch.
- Q094 Write a decorator that measures execution time.
- Q095 Implement the Singleton design pattern.
- Q096 Write a function to deep clone a nested dict.
- Q097 Find the first non-repeating character in a string.
- Q098 Implement a basic trie (prefix tree).
- Q099 Flatten a deeply nested dictionary.
- Q100 Implement an iterator class that generates perfec...

BASIC

10 questions

Q1

BASIC

What is Python?

ANSWER

Python is a high-level, interpreted, general-purpose programming language emphasising readability and simplicity. It supports multiple paradigms — procedural, object-oriented, and functional — and comes with an extensive standard library.

Q2

BASIC

What are Python's key features?

ANSWER

Interpreted (no compilation step), dynamically typed, garbage-collected memory, rich standard library, platform-independent, supports OOP and functional styles, and an active open-source ecosystem.

Q3

BASIC

What is PEP 8?

ANSWER

PEP 8 is Python's official style guide. It recommends 4-space indentation, snake_case for variables and functions, PascalCase for classes, max line length of 79 chars, and clear docstrings.

Q4

BASIC

What are Python's built-in data types?

ANSWER

int, float, complex, str, bool, list, tuple, set, frozenset, dict, bytes, bytearray, NoneType.

EXAMPLE

```
x = 42          # int
y = 3.14        # float
z = 'hello'     # str
a = [1,2,3]     # list
b = (1,2,3)     # tuple
c = {1,2,3}     # set
d = {'k': 'v'}  # dict
```

Q5

BASIC

What is the difference between a list and a tuple?

ANSWER

Lists are mutable (can be changed after creation) and use square brackets. Tuples are immutable and use parentheses. Tuples are faster for iteration and can be used as dict keys.

EXAMPLE

```
lst = [1, 2, 3]
lst[0] = 99      # OK – list is mutable

tup = (1, 2, 3)
tup[0] = 99      # TypeError – tuple is immutable
```

Q6

BASIC

What is a dictionary in Python?

ANSWER

A dict is an ordered (Python 3.7+) collection of key-value pairs. Keys must be hashable and unique; values can be any object.

EXAMPLE

```
d = {'name': 'Alice', 'age': 30}
print(d['name'])    # Alice
d['city'] = 'NY'
print(list(d.keys())) # ['name', 'age', 'city']
```

Q7

BASIC

What are Python sets?

ANSWER

Sets are unordered collections of unique, hashable elements. They support fast $O(1)$ membership tests and set-algebra operations (union, intersection, difference).

EXAMPLE

```
s = {1, 2, 3, 2, 1}
print(s)          # {1, 2, 3}
print(2 in s)      # True
print(s | {4, 5})  # {1, 2, 3, 4, 5}
```

Q8

BASIC

How do you swap two variables in Python?

ANSWER

Python allows elegant tuple unpacking to swap variables in one line — no temporary variable needed.

EXAMPLE

```
a, b = 5, 10
a, b = b, a
print(a, b)    # 10 5
```

Q9

BASIC

What is the difference between '==' and 'is'?

ANSWER

'==' checks value equality (do they hold the same data?). 'is' checks identity (are they the exact same object in memory?).

EXAMPLE

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)    # True  - same values
print(a is b)    # False - different objects
c = a
print(a is c)    # True  - same object
```

Q10

BASIC

What are Python keywords?

ANSWER

Reserved words that cannot be used as identifiers. Use `keyword.kwlist` to list all of them.

EXAMPLE

```
import keyword
print(keyword.kwlist)
# ['False', 'None', 'True', 'and', 'as',
#  'assert', 'async', 'await', 'break', ...]
```

CONCEPT

15 questions

Q11

CONCEPT

What is a Python decorator?

ANSWER

A decorator is a function that wraps another function, adding behaviour before or after without modifying the original function body. Applied with the `@` syntax.

EXAMPLE

```
def log(func):
    def wrapper(*args, **kwargs):
        print(f'Calling {func.__name__}')
        return func(*args, **kwargs)
    return wrapper

@log
def greet(name):
    return f'Hello, {name}'

greet('Alice') # prints: Calling greet
```

Q12

CONCEPT

What is a Python generator?

ANSWER

A generator is a function that uses 'yield' to produce values one at a time, pausing execution between calls. Values are produced lazily — ideal for large or infinite sequences.

EXAMPLE

```
def count_up(n):
    i = 0
    while i < n:
        yield i
        i += 1

for x in count_up(3):
    print(x)    # 0, then 1, then 2
```

Q13

CONCEPT

What is a list comprehension?

ANSWER

A compact, readable syntax for building lists by iterating and optionally filtering elements in a single expression.

EXAMPLE

```
squares = [x**2 for x in range(6)]
# [0, 1, 4, 9, 16, 25]

evens = [x for x in range(10) if x % 2 == 0]
# [0, 2, 4, 6, 8]
```

Q14

CONCEPT

What are *args and **kwargs?

ANSWER

*args collects extra positional arguments into a tuple. **kwargs collects extra keyword arguments into a dict. Both allow functions to accept a variable number of arguments.

EXAMPLE

```
def show(*args, **kwargs):
    print(args)    # (1, 2, 3)
    print(kwargs)  # {'a': 10, 'b': 20}

show(1, 2, 3, a=10, b=20)
```

Q15 **CONCEPT** What is a lambda function?**ANSWER**

A lambda is an anonymous, single-expression function defined inline. Useful for short callbacks, sorting keys, or anywhere a full def would be unnecessarily verbose.

EXAMPLE

```
double = lambda x: x * 2
print(double(5))    # 10

words = ['banana', 'apple', 'cherry']
print(sorted(words, key=lambda w: len(w)))
# ['apple', 'banana', 'cherry']
```

Q16 **CONCEPT** What is the difference between shallow copy and deep copy?**ANSWER**

A shallow copy creates a new outer container but still references the same nested objects. A deep copy recursively duplicates everything — completely independent.

EXAMPLE

```
import copy
orig = [[1, 2], [3, 4]]
shallow = copy.copy(orig)
deep = copy.deepcopy(orig)
orig[0][0] = 99
print(shallow[0][0]) # 99 (shared inner list)
print(deep[0][0])    # 1 (independent copy)
```

Q17 **CONCEPT** What is Python's GIL?**ANSWER**

The Global Interpreter Lock (GIL) is a mutex in CPython that allows only one thread to execute Python bytecode at a time. This prevents true multi-core CPU parallelism in threads. Use multiprocessing or asyncio for concurrency instead.

EXAMPLE

```
# CPU-bound: use multiprocessing, not threading
from multiprocessing import Pool
def square(x): return x * x
with Pool(4) as p:
    print(p.map(square, range(8)))
```


Q18

CONCEPT

What are Python's scope rules (LEGB)?**ANSWER**

Python looks up names in four scopes in order: Local -> Enclosing -> Global -> Built-in. This is the LEGB rule.

EXAMPLE

```
x = 'global'
def outer():
    x = 'enclosing'
    def inner():
        x = 'local'
        print(x)    # local
    inner()
outer()
```

Q19

CONCEPT

What is a closure?**ANSWER**

A closure is a nested function that captures and remembers variables from its enclosing scope, even after the outer function has returned.

EXAMPLE

```
def make_adder(n):
    def adder(x):
        return x + n    # 'n' is captured
    return adder

add5 = make_adder(5)
print(add5(3))    # 8
print(add5(10))   # 15
```

Q20

CONCEPT

What is monkey patching?**ANSWER**

Monkey patching means dynamically replacing or extending a class/module's attributes or methods at runtime, without modifying the original source code. Common in tests and hot-fixes.

EXAMPLE

```
class Dog:
    def speak(self): return 'Woof'

def cat_speak(self): return 'Meow'

Dog.speak = cat_speak
print(Dog().speak())    # Meow
```

Q21

CONCEPT

What is the difference between `__str__` and `__repr__`?

ANSWER

`__repr__` returns an unambiguous developer-facing string (ideally eval-able back to the object). `__str__` returns a human-readable string. `print()` uses `__str__`; the REPL uses `__repr__`.

EXAMPLE

```
class Point:
    def __init__(self, x, y): self.x, self.y = x, y
    def __repr__(self): return f'Point({self.x}, {self.y})'
    def __str__(self): return f'({self.x}, {self.y})'

p = Point(1, 2)
print(str(p))    # (1, 2)
print(repr(p))   # Point(1, 2)
```

Q22

CONCEPT

What is method resolution order (MRO)?

ANSWER

MRO defines the class lookup order for methods in an inheritance hierarchy. Python uses the C3 linearisation algorithm, viewable via `Class.__mro__`.

EXAMPLE

```
class A: pass
class B(A): pass
class C(A): pass
class D(B, C): pass

print([c.__name__ for c in D.__mro__])
# ['D', 'B', 'C', 'A', 'object']
```

Q23

CONCEPT

What are Python's dunder (magic) methods?

ANSWER

Double-underscore methods define how objects behave with built-in operations. Examples: `__init__` (construction), `__len__` (`len()`), `__add__` (+), `__iter__` (iteration), `__enter__`/`__exit__` (with statement).

EXAMPLE

```
class MyList:
    def __init__(self, data): self.data = data
    def __len__(self): return len(self.data)
    def __getitem__(self, i): return self.data[i]

ml = MyList([10, 20, 30])
print(len(ml))    # 3
print(ml[1])      # 20
```

Q24

CONCEPT

What is a context manager?

ANSWER

A context manager handles resource setup (`__enter__`) and guaranteed cleanup (`__exit__`), used with the 'with' statement. Ensures resources are released even if an exception occurs.

EXAMPLE

```
class Timer:
    def __enter__(self):
        import time; self.t = time.time(); return self
    def __exit__(self, *a):
        import time
        print(f'Elapsed: {time.time()-self.t:.4f}s')

with Timer():
    sum(range(1_000_000))
```

Q25

CONCEPT

What is the difference between staticmethod and classmethod?

ANSWER

@staticmethod is a plain function inside a class — no implicit first argument. @classmethod receives the class (cls) and can access or modify class-level state.

EXAMPLE

```
class Counter:
    count = 0
    @staticmethod
    def description(): return 'A counter class'
    @classmethod
    def increment(cls): cls.count += 1

Counter.increment()
print(Counter.count)      # 1
print(Counter.description()) # A counter class
```

INTERMEDIATE

15 questions

Q26

INTERMEDIATE

How does Python manage memory?

ANSWER

Python uses reference counting as its primary memory management mechanism. When an object's reference count drops to zero it is freed immediately. A cyclic garbage collector handles reference cycles that reference counting cannot detect.

EXAMPLE

```
import sys
a = []
print(sys.getrefcount(a)) # 2 (a + getrefcount arg)
b = a
print(sys.getrefcount(a)) # 3
```

Q27

INTERMEDIATE

What is the difference between mutable and immutable objects?

ANSWER

Mutable objects (list, dict, set) can be changed in-place. Immutable objects (int, str, tuple, frozenset) cannot; any 'modification' creates a new object instead.

EXAMPLE

```
s = 'hello'
id1 = id(s)
s += ' world' # new object created
print(id(s) == id1) # False

lst = [1, 2]
id1 = id(lst)
lst.append(3) # modified in-place
print(id(lst) == id1) # True
```

Q28

INTERMEDIATE

What are iterators and iterables?

ANSWER

An iterable is any object with `__iter__` that returns an iterator. An iterator has both `__iter__` and `__next__`. All iterators are iterables, but not all iterables are iterators.

EXAMPLE

```
nums = [1, 2, 3] # iterable
it = iter(nums) # iterator
print(next(it)) # 1
print(next(it)) # 2
print(next(it)) # 3
next(it)        # raises StopIteration
```

Q29

INTERMEDIATE

What is the difference between map, filter, and reduce?

ANSWER

map() transforms each element. filter() keeps only elements where the function returns True. reduce() (functools) collapses a sequence to a single accumulated value.

EXAMPLE

```
from functools import reduce
nums = [1, 2, 3, 4, 5]
list(map(lambda x: x**2, nums))      # [1,4,9,16,25]
list(filter(lambda x: x%2==0, nums)) # [2, 4]
reduce(lambda a, b: a+b, nums)      # 15
```

Q30

INTERMEDIATE

What is exception handling in Python?

ANSWER

Python uses try/except/else/finally blocks. The 'try' block contains risky code; 'except' catches specific errors; 'else' runs only when no exception occurs; 'finally' always runs for cleanup.

EXAMPLE

```
try:
    result = 10 / int(input())
except ZeroDivisionError:
    print('Cannot divide by zero')
except ValueError as e:
    print(f'Bad input: {e}')
else:
    print(f'Result: {result}')
finally:
    print('Done')
```

Q31

INTERMEDIATE

How do you create a custom exception?

ANSWER

Subclass Exception (or a more specific built-in). Override __init__ to accept custom arguments and pass a message to super().__init__.

EXAMPLE

```
class InsufficientFunds(Exception):
    def __init__(self, needed, available):
        self.needed = needed
        self.available = available
        super().__init__(
            f'Need {needed}, only {available} available')

raise InsufficientFunds(100, 50)
```

Q32

INTERMEDIATE

What is multiple inheritance? Explain the diamond problem.**ANSWER**

Multiple inheritance allows a class to inherit from more than one parent. The 'diamond problem' occurs when two parents share a common ancestor. Python's C3 MRO resolves this consistently.

EXAMPLE

```
class A:
    def hello(self): print('A')
class B(A):
    def hello(self): print('B')
class C(A):
    def hello(self): print('C')
class D(B, C): pass
D().hello()    # B (MRO: D->B->C->A)
```

Q33

INTERMEDIATE

What are Python's file I/O modes?**ANSWER**

'r' = read, 'w' = write (truncate), 'a' = append, 'x' = create (error if exists), 'b' = binary flag, '+' = read+write. Combine freely: 'rb', 'w+', 'ab', etc.

EXAMPLE

```
with open('notes.txt', 'w') as f:
    f.write('Hello, World!')

with open('notes.txt', 'r') as f:
    print(f.read())    # Hello, World!
```

Q34

INTERMEDIATE

What is pickling in Python?**ANSWER**

Pickling serialises any Python object to a byte stream so it can be saved to disk or sent over a network. Unpickling restores the original object. Use the 'pickle' module.

EXAMPLE

```
import pickle

data = {'scores': [10, 20, 30], 'name': 'Alice'}

with open('data.pkl', 'wb') as f:
    pickle.dump(data, f)

with open('data.pkl', 'rb') as f:
    loaded = pickle.load(f)
print(loaded['name'])    # Alice
```

Q35

INTERMEDIATE

What is the difference between pass, continue, and break?**ANSWER**

'pass' is a no-op placeholder used where syntax requires a statement. 'continue' skips the rest of the current loop body and moves to the next iteration. 'break' exits the loop entirely.

EXAMPLE

```
for i in range(6):
    if i == 2: continue    # skip 2
    if i == 5: break      # stop before 5
    print(i)              # prints 0, 1, 3, 4
```

Q36

INTERMEDIATE

What are Python modules and packages?**ANSWER**

A module is a single .py file. A package is a directory containing an __init__.py that groups related modules. Import with 'import module' or 'from package.module import name'.

EXAMPLE

```
# mypackage/
#  __init__.py
#  utils.py

from mypackage.utils import helper
result = helper('data')
```

Q37

INTERMEDIATE

What is __init__.py?**ANSWER**

A file that marks a directory as a Python package so its modules can be imported. It can be empty or contain package-level initialisation code. Optional for namespace packages (Python 3.3+).

EXAMPLE

```
# project/
#  mypackage/
#    __init__.py  <- makes it a package
#    math_utils.py
#    string_utils.py
```

Q38

INTERMEDIATE

What is the 'with' statement?

ANSWER

The 'with' statement invokes a context manager — calling `__enter__` on entry and `__exit__` on exit — ensuring resources (files, locks, connections) are always released, even if an exception is raised.

EXAMPLE

```
# Without 'with' - error-prone
f = open('file.txt')
data = f.read()  # if this fails, file stays open
f.close()

# With 'with' - always closed
with open('file.txt') as f:
    data = f.read()
```

Q39

INTERMEDIATE

What is string formatting in Python?

ANSWER

Three main approaches: %-formatting (legacy), `str.format()` (Python 2.6+), and f-strings (Python 3.6+, fastest and most readable).

EXAMPLE

```
name, score = 'Alice', 98.5

# f-string (preferred)
print(f'{name} scored {score:.1f}')

# str.format()
print('{} scored {:.1f}'.format(name, score))

# % operator (legacy)
print('%s scored %.1f' % (name, score))
```

Q40

INTERMEDIATE

What are Python's built-in functions for sequences?

ANSWER

Key built-ins: `len()`, `sum()`, `min()`, `max()`, `sorted()`, `reversed()`, `enumerate()`, `zip()`, `any()`, `all()`, `map()`, `filter()`. These cover the vast majority of sequence-processing needs.

EXAMPLE

```
data = [3, 1, 4, 1, 5, 9, 2, 6]
print(sum(data))          # 31
print(sorted(data)[:3])   # [1, 1, 2]
for i, v in enumerate(['a', 'b', 'c']):
    print(i, v)           # 0 a, 1 b, 2 c
```


ADVANCED

10 questions

Q41

ADVANCED

What is async/await in Python?

ANSWER

async/await enables cooperative concurrency via coroutines and an event loop (asyncio). 'async def' defines a coroutine. 'await' yields control until the awaitable finishes, letting other tasks run meanwhile.

EXAMPLE

```
import asyncio

async def fetch(name, delay):
    await asyncio.sleep(delay)
    print(f'{name} done')

async def main():
    await asyncio.gather(
        fetch('Task A', 2),
        fetch('Task B', 1),
    )

asyncio.run(main())
# Task B done (after 1s)
# Task A done (after 2s)
```

Q42

ADVANCED

What are metaclasses in Python?

ANSWER

A metaclass is the class of a class — it controls how a class itself is created. 'type' is the default metaclass. Custom metaclasses intercept `__new__` or `__init__` to modify class creation.

EXAMPLE

```
class UpperMeta(type):
    def __new__(mcs, name, bases, ns):
        upper_ns = {k.upper(): v for k, v in ns.items()
                     if not k.startswith('_')}
        return super().__new__(mcs, name, bases, upper_ns)

class Config(metaclass=UpperMeta):
    host = 'localhost'
    port = 8080
```

Q43

ADVANCED

What are descriptors?

ANSWER

A descriptor is an object that defines `__get__`, `__set__`, or `__delete__` and is assigned as a class attribute. It controls attribute access for all instances. Properties, classmethods, and staticmethods are all built on descriptors.

EXAMPLE

```
class Positive:
    def __set_name__(self, owner, name): self.name = name
    def __get__(self, obj, tp):
        return obj.__dict__.get(self.name)
    def __set__(self, obj, val):
        if val < 0: raise ValueError('Must be positive')
        obj.__dict__[self.name] = val

class Circle:
    radius = Positive()
```

Q44

ADVANCED

Explain Python's dataclasses.

ANSWER

`@dataclass` (Python 3.7+) auto-generates `__init__`, `__repr__`, and `__eq__` from class-level type annotations. Optional flags add `__hash__`, ordering methods, frozen (immutable) instances, and more.

EXAMPLE

```
from dataclasses import dataclass, field

@dataclass(order=True)
class Employee:
    name: str
    salary: float = 50_000.0
    skills: list = field(default_factory=list)

e = Employee('Alice', 75_000)
print(e) # Employee(name='Alice', salary=75000.0, ...)
```

Q45

ADVANCED

What is Python's abc module?

ANSWER

The abc module provides Abstract Base Classes (ABC). Marking a method with `@abstractmethod` forces every concrete subclass to implement it, otherwise instantiation raises `TypeError`.

EXAMPLE

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self) -> float: ...
    @abstractmethod
    def perimeter(self) -> float: ...

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return 3.14159 * self.r**2
    def perimeter(self): return 2 * 3.14159 * self.r
```

Q46

ADVANCED

What is the difference between threading and multiprocessing?

ANSWER

threading shares memory but is GIL-limited for CPU-bound tasks — good for I/O-bound work.
multiprocessing spawns separate OS processes with their own memory and GIL — ideal for CPU-bound parallelism.

EXAMPLE

```
from multiprocessing import Pool

def cpu_task(n):
    return sum(i*i for i in range(n))

with Pool(4) as p:
    results = p.map(cpu_task, [10**6]*4)
print(results)
```

Q47

ADVANCED

What are `__slots__` in Python?

ANSWER

`__slots__` replaces the per-instance `__dict__` with a fixed tuple of attribute names. This saves memory (no dict overhead) and slightly speeds up attribute access — important when creating millions of instances.

EXAMPLE

```
class Point:
    __slots__ = ('x', 'y')
    def __init__(self, x, y):
        self.x, self.y = x, y

p = Point(3, 4)
print(p.x)      # 3
p.z = 5         # AttributeError – not in __slots__
```

Q48

ADVANCED

What is `functools.lru_cache`?

ANSWER

`lru_cache` is a decorator that memoises a function's results in a Least Recently Used cache. On repeated calls with the same arguments, it returns the cached value instead of recomputing — great for pure functions.

EXAMPLE

```
from functools import lru_cache

@lru_cache(maxsize=None) # unlimited cache
def fib(n):
    if n < 2: return n
    return fib(n-1) + fib(n-2)

print(fib(50))          # 12586269025 – instant
print(fib.cache_info())
```

Q49

ADVANCED

What are Python type hints?**ANSWER**

Type hints (PEP 484, Python 3.5+) annotate variables and function signatures with expected types. They don't affect runtime behaviour but enable static analysis tools (mypy, Pyright, Pylance) to catch bugs early.

EXAMPLE

```
from typing import Optional, List, Dict

def find_user(
    users: List[Dict[str, str]],
    name: str
) -> Optional[Dict[str, str]]:
    for u in users:
        if u['name'] == name: return u
    return None
```

Q50

ADVANCED

What is memoisation and how do you implement it?**ANSWER**

Memoisation caches the return values of expensive function calls so they are only computed once per unique input. Implement with a plain dict or use `@functools.lru_cache` for a ready-made solution.

EXAMPLE

```
cache = {}

def expensive(n):
    if n in cache:
        return cache[n]
    result = sum(i**2 for i in range(n)) # slow
    cache[n] = result
    return result

expensive(10_000) # computed
expensive(10_000) # returned from cache
```

CODING PROBLEM

50 questions

Q51

CODING PROBLEM

Reverse a string without using slicing.

ANSWER

Iterate the string from left to right, prepending each character to the result. Alternatively use `".join(reversed(s))` for a Pythonic $O(n)$ approach.

EXAMPLE

```
def reverse_str(s):
    result = ''
    for ch in s:
        result = ch + result
    return result

print(reverse_str('hello'))    # olleh

# Pythonic one-liner:
print(''.join(reversed('hello')))
```

Q52

CODING PROBLEM

Check if a string is a palindrome.

ANSWER

Strip and lowercase the string, then compare it to its own reverse. A palindrome reads the same forwards and backwards.

EXAMPLE

```
def is_palindrome(s):
    s = s.lower().replace(' ', '')
    return s == s[::-1]

print(is_palindrome('racecar'))    # True
print(is_palindrome('A man a plan')) # False
print(is_palindrome('level'))      # True
```

Q53

CODING PROBLEM

Find the factorial of a number.

ANSWER

$\text{Factorial}(n) = n * (n-1) * \dots * 1$. Implement recursively (elegant) or iteratively (no stack overflow risk). `math.factorial()` is also available.

EXAMPLE

```
def factorial(n):
    if n <= 1: return 1
    return n * factorial(n - 1)

print(factorial(0)) # 1
print(factorial(5)) # 120
print(factorial(10)) # 3628800
```

Q54

CODING PROBLEM

Print the Fibonacci sequence up to n terms.

ANSWER

Maintain two running variables a and b; each step yields a and then advances both by one Fibonacci step.

EXAMPLE

```
def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        print(a, end=' ')
        a, b = b, a + b

fibonacci(8)
# 0 1 1 2 3 5 8 13
```

Q55

CODING PROBLEM

Find duplicate elements in a list.

ANSWER

Track all seen elements in one set. When an element appears in 'seen' a second time, add it to the 'dupes' set.

EXAMPLE

```
def find_duplicates(lst):
    seen, dupes = set(), set()
    for x in lst:
        if x in seen:
            dupes.add(x)
        seen.add(x)
    return list(dupes)

print(find_duplicates([1,2,3,2,4,3,5]))
# [2, 3]
```

Q56

CODING PROBLEM

Count the occurrences of each element in a list.

ANSWER

collections.Counter builds a frequency map in one pass and offers convenient methods like most_common().

EXAMPLE

```
from collections import Counter

data = ['a', 'b', 'a', 'c', 'b', 'a']
c = Counter(data)
print(c)           # Counter({'a':3, 'b':2, 'c':1})
print(c.most_common(2)) # [('a',3), ('b',2)]
```

Q57

CODING PROBLEM

Flatten a nested list.

ANSWER

Recursively walk each element. If it is a list, recurse into it; otherwise append it to the result.

EXAMPLE

```
def flatten(lst):
    result = []
    for item in lst:
        if isinstance(item, list):
            result.extend(flatten(item))
        else:
            result.append(item)
    return result

print(flatten([1, [2, [3, 4]], 5]))
# [1, 2, 3, 4, 5]
```

Q58

CODING PROBLEM

Implement binary search.

ANSWER

Repeatedly halve the search space by comparing the middle element to the target. Returns the index or -1. Requires a sorted input array.

EXAMPLE

```
def binary_search(arr, target):
    lo, hi = 0, len(arr) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if arr[mid] == target: return mid
        elif arr[mid] < target: lo = mid + 1
        else: hi = mid - 1
    return -1

print(binary_search([1, 3, 5, 7, 9], 5)) # 2
```


Q59

CODING PROBLEM

Implement bubble sort.

ANSWER

Repeatedly pass through the list swapping adjacent out-of-order elements. Each full pass 'bubbles' the largest unsorted element to its final position.

EXAMPLE

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(n - i - 1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr

print(bubble_sort([64,34,25,12,22]))
# [12, 22, 25, 34, 64]
```

Q60

CODING PROBLEM

Find two numbers that sum to a target.

ANSWER

Use a hash map to store each number's index. For every element, check whether its complement (target - element) has already been seen.

EXAMPLE

```
def two_sum(nums, target):
    seen = {}
    for i, n in enumerate(nums):
        complement = target - n
        if complement in seen:
            return [seen[complement], i]
        seen[n] = i
    return []

print(two_sum([2,7,11,15], 9)) # [0, 1]
```

Q61

CODING PROBLEM

Check if two strings are anagrams.

ANSWER

Two strings are anagrams if they share the exact same character multiset. Compare sorted versions or use Counter for an $O(n)$ solution.

EXAMPLE

```
from collections import Counter

def is_anagram(s1, s2):
    return Counter(s1.lower()) == Counter(s2.lower())

print(is_anagram('listen', 'silent')) # True
print(is_anagram('hello', 'world'))  # False
```

Q62

CODING PROBLEM

Find the most frequent element in a list.

ANSWER

Counter.most_common(1) returns a list with the single (element, count) pair for the most frequent item.

EXAMPLE

```
from collections import Counter

def most_frequent(lst):
    return Counter(lst).most_common(1)[0][0]

print(most_frequent([1,2,2,3,3,3,4]))
# 3
```

Q63

CODING PROBLEM

Merge two sorted lists.

ANSWER

The two-pointer technique compares heads of both lists and picks the smaller one each step. $O(n+m)$ time, $O(n+m)$ space.

EXAMPLE

```
def merge_sorted(a, b):
    result, i, j = [], 0, 0
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            result.append(a[i]); i += 1
        else:
            result.append(b[j]); j += 1
    return result + a[i:] + b[j:]

print(merge_sorted([1,3,5],[2,4,6]))
# [1, 2, 3, 4, 5, 6]
```

Q64

CODING PROBLEM

Implement a stack using a list.

ANSWER

Python lists provide $O(1)$ append (push) and pop from the end. Wrap them in a class for a clean interface.

EXAMPLE

```
class Stack:
    def __init__(self):    self._data = []
    def push(self, val):   self._data.append(val)
    def pop(self):        return self._data.pop()
    def peek(self):       return self._data[-1]
    def is_empty(self):   return not self._data

s = Stack()
s.push(1); s.push(2); s.push(3)
print(s.pop())    # 3
```

Q65

CODING PROBLEM

Implement a queue using collections.deque.

ANSWER

deque (double-ended queue) provides $O(1)$ append (enqueue at right) and popleft (dequeue from left) — unlike list which is $O(n)$ for popleft.

EXAMPLE

```
from collections import deque

class Queue:
    def __init__(self):    self._q = deque()
    def enqueue(self, val): self._q.append(val)
    def dequeue(self):     return self._q.popleft()
    def is_empty(self):    return not self._q

q = Queue()
q.enqueue('a'); q.enqueue('b')
print(q.dequeue())    # a
```

Q66

CODING PROBLEM

Count vowels in a string.

ANSWER

Iterate each character and check membership against a vowel set. Sets offer $O(1)$ lookup vs $O(n)$ for a string.

EXAMPLE

```
def count_vowels(s):
    vowels = set('aeiouAEIOU')
    return sum(1 for c in s if c in vowels)

print(count_vowels('Hello World'))    # 3
print(count_vowels('Python'))        # 1
```

Q67

CODING PROBLEM

Find largest and smallest in a list without built-ins.

ANSWER

Initialise both min and max to the first element, then update them in a single pass.

EXAMPLE

```
def min_max(lst):  
    mn = mx = lst[0]  
    for x in lst[1:]:  
        if x < mn: mn = x  
        if x > mx: mx = x  
    return mn, mx  
  
print(min_max([3,1,4,1,5,9,2,6]))  
# (1, 9)
```

Q68

CODING PROBLEM

Remove duplicates from a list preserving order.

ANSWER

Use an insertion-order-preserving set via a 'seen' set and a result list. `dict.fromkeys()` also works since Python 3.7+.

EXAMPLE

```
def remove_dupes(lst):  
    seen, result = set(), []  
    for x in lst:  
        if x not in seen:  
            result.append(x); seen.add(x)  
    return result  
  
print(remove_dupes([1,2,3,2,1,4]))  
# [1, 2, 3, 4]
```

Q69

CODING PROBLEM

Implement a function to check for balanced brackets.

ANSWER

Push opening brackets onto a stack. For each closing bracket, verify the top of the stack holds the matching opener.

EXAMPLE

```
def is_balanced(s):
    stack = []
    match = {'(': '(', ')': ')', '[': '[', ']'': ']' }
    for ch in s:
        if ch in '([{': stack.append(ch)
        elif ch in ')]}':
            if not stack or stack[-1] != match[ch]:
                return False
            stack.pop()
    return not stack

print(is_balanced('{[]}')) # True
print(is_balanced('([])')) # False
```

Q70

CODING PROBLEM

Generate all permutations of a string.

ANSWER

itertools.permutations yields all orderings as tuples; join each to get strings. For a manual approach, fix one character and recurse on the rest.

EXAMPLE

```
from itertools import permutations

def all_perms(s):
    return sorted(''.join(p) for p in permutations(s))

print(all_perms('abc'))
# ['abc', 'acb', 'bac', 'bca', 'cab', 'cba']
```

Q71

CODING PROBLEM

Implement merge sort.

ANSWER

Divide the array in half, recursively sort each half, then merge the two sorted halves. $O(n \log n)$ time, $O(n)$ space.

EXAMPLE

```
def merge_sort(arr):
    if len(arr) <= 1: return arr
    mid = len(arr) // 2
    L, R = merge_sort(arr[:mid]), merge_sort(arr[mid:])
    result, i, j = [], 0, 0
    while i < len(L) and j < len(R):
        if L[i] <= R[j]: result.append(L[i]); i += 1
        else:           result.append(R[j]); j += 1
    return result + L[i:] + R[j:]
```

Q72

CODING PROBLEM

Find common elements between two lists.

ANSWER

Convert both lists to sets and use the & intersection operator. $O(\min(n,m))$ average time.

EXAMPLE

```
def common(a, b):
    return list(set(a) & set(b))

print(common([1,2,3,4],[3,4,5,6]))
# [3, 4]
```

Q73

CODING PROBLEM

Implement a simple linked list.

ANSWER

A Node holds data and a 'next' pointer. LinkedList maintains a head and provides append, which walks to the tail.

EXAMPLE

```
class Node:
    def __init__(self, data): self.data=data; self.next=None

class LinkedList:
    def __init__(self): self.head = None
    def append(self, data):
        new = Node(data)
        if not self.head: self.head=new; return
        cur = self.head
        while cur.next: cur = cur.next
        cur.next = new
    def to_list(self):
        vals, cur = [], self.head
        while cur: vals.append(cur.data); cur=cur.next
        return vals
```

Q74

CODING PROBLEM

Check if a number is prime.

ANSWER

A prime has no divisors other than 1 and itself. Trial division up to $\sqrt{n}$ — skipping even numbers after 2 — is sufficient.

EXAMPLE

```
def is_prime(n):
    if n < 2:      return False
    if n == 2:     return True
    if n % 2 == 0: return False
    for i in range(3, int(n**0.5)+1, 2):
        if n % i == 0: return False
    return True

primes = [x for x in range(2,20) if is_prime(x)]
print(primes)  # [2,3,5,7,11,13,17,19]
```

Q75

CODING PROBLEM

Calculate the GCD of two numbers.

ANSWER

Euclid's algorithm: $\text{gcd}(a,b) = \text{gcd}(b, a\%b)$. Repeat until $b == 0$; the answer is a . Also available as `math.gcd()`.

EXAMPLE

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

print(gcd(48, 18))    # 6
print(gcd(100, 75))  # 25

from math import gcd as math_gcd
print(math_gcd(48, 18)) # 6
```

Q76

CODING PROBLEM

Rotate a list by k positions.

ANSWER

Slice the last k elements and prepend them to the rest. Use modulo to handle $k > \text{len}(\text{lst})$.

EXAMPLE

```
def rotate(lst, k):
    if not lst: return lst
    k = k % len(lst)
    return lst[-k:] + lst[:-k]

print(rotate([1,2,3,4,5], 2))
# [4, 5, 1, 2, 3]
```

Q77

CODING PROBLEM

Find the missing number in 1..n.

ANSWER

The expected sum of 1..n is $n*(n+1)//2$. Subtract the actual sum to find the missing value in $O(n)$ time and $O(1)$ space.

EXAMPLE

```
def missing_number(lst):
    n = len(lst) + 1
    expected = n * (n + 1) // 2
    return expected - sum(lst)

print(missing_number([1,2,4,5,6])) # 3
print(missing_number([2,3,4,5,6])) # 1
```


Q78

CODING PROBLEM

Implement a Caesar cipher.

ANSWER

Shift each alphabetic character by 'shift' positions using modular arithmetic (mod 26), preserving case and leaving non-alpha characters unchanged.

EXAMPLE

```
def caesar(text, shift):
    result = []
    for ch in text:
        if ch.isalpha():
            base = ord('A') if ch.isupper() else ord('a')
            result.append(chr((ord(ch)-base+shift)%26+base))
        else:
            result.append(ch)
    return ''.join(result)

print(caesar('Hello, World!', 3)) # Koor, Zruog!
```

Q79

CODING PROBLEM

Write a function to transpose a matrix.

ANSWER

zip(*matrix) unpacks rows and re-zips them as columns. Wrap each result in list() to get a plain list of lists.

EXAMPLE

```
def transpose(matrix):
    return [list(row) for row in zip(*matrix)]

m = [[1,2,3],[4,5,6]]
print(transpose(m))
# [[1, 4], [2, 5], [3, 6]]
```

Q80

CODING PROBLEM

Implement a function that returns the power set.

ANSWER

The power set of a list contains all possible subsets including the empty set. itertools.combinations generates subsets of each size k from 0 to n.

EXAMPLE

```
from itertools import combinations

def power_set(lst):
    result = []
    for r in range(len(lst) + 1):
        result.extend(combinations(lst, r))
    return result

print(power_set([1,2,3]))
# [(), (1,), (2,), (3,), (1,2), (1,3), (2,3), (1,2,3)]
```

Q81

CODING PROBLEM

Count words in a string.

ANSWER

`str.split()` with no argument splits on any whitespace and removes empty tokens, cleanly handling multiple spaces, tabs, and newlines.

EXAMPLE

```
def word_count(text):
    return len(text.split())

print(word_count(' Hello World Python '))
# 3

# For per-word counts:
from collections import Counter
print(Counter('the cat sat on the mat'.split()))
```

Q82

CODING PROBLEM

Find the longest common prefix in a list of strings.

ANSWER

Sort the list — the longest common prefix can only differ between the lexicographically first and last strings. Compare those two character by character.

EXAMPLE

```
def lcp(strs):
    if not strs: return ''
    strs.sort()
    prefix = []
    for a, b in zip(strs[0], strs[-1]):
        if a != b: break
        prefix.append(a)
    return ''.join(prefix)

print(lcp(['flower', 'flow', 'flight'])) # 'fl'
```

Q83

CODING PROBLEM

Implement quick sort.

ANSWER

Choose a pivot, partition elements into less-than, equal, and greater-than groups, then recursively sort the outer groups.

EXAMPLE

```
def quick_sort(arr):
    if len(arr) <= 1: return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    mid = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + mid + quick_sort(right)

print(quick_sort([3,6,8,10,1,2,1]))
# [1, 1, 2, 3, 6, 8, 10]
```

Q84

CODING PROBLEM

Find all prime factors of a number.

ANSWER

Divide out all 2s first, then try odd divisors up to $\sqrt{n}$. Any remainder > 1 at the end is itself a prime factor.

EXAMPLE

```
def prime_factors(n):
    factors = []
    while n % 2 == 0:
        factors.append(2); n //= 2
    i = 3
    while i * i <= n:
        while n % i == 0:
            factors.append(i); n //= i
        i += 2
    if n > 2: factors.append(n)
    return factors

print(prime_factors(360)) # [2,2,2,3,3,5]
```

Q85

CODING PROBLEM

Convert decimal to binary without bin().

ANSWER

Repeatedly divide by 2 and collect remainders. Reverse the list of remainders to get the binary representation.

EXAMPLE

```
def to_binary(n):
    if n == 0: return '0'
    bits = []
    while n:
        bits.append(str(n % 2))
        n //= 2
    return ''.join(reversed(bits))

print(to_binary(0))    # 0
print(to_binary(10))   # 1010
print(to_binary(42))   # 101010
```

Q86

CODING PROBLEM

Implement a basic expression calculator (+/-).

ANSWER

Use a stack and a running number. Flush to the stack on each operator, then sum the stack at the end.

EXAMPLE

```
def calculate(s):
    stack, num, sign = [], 0, 1
    for ch in s + '+':
        if ch.isdigit():
            num = num * 10 + int(ch)
        elif ch in '+-':
            stack.append(sign * num)
            num = 0
            sign = 1 if ch == '+' else -1
    return sum(stack)

print(calculate('12+3-5')) # 10
```

Q87

CODING PROBLEM

Find the intersection of two sorted arrays.

ANSWER

Two-pointer technique: advance the pointer pointing at the smaller value; when equal, record the match.
 $O(n+m)$ time, $O(1)$ extra space.

EXAMPLE

```
def intersect(a, b):
    i, j, result = 0, 0, []
    while i < len(a) and j < len(b):
        if a[i] == b[j]: result.append(a[i]); i+=1; j+=1
        elif a[i] < b[j]: i += 1
        else: j += 1
    return result

print(intersect([1,2,4,5],[2,4,6,8]))
# [2, 4]
```

Q88

CODING PROBLEM

Implement strlen without len().

ANSWER

Count each character in a for loop. This demonstrates iterator protocol under the hood.

EXAMPLE

```
def strlen(s):
    count = 0
    for _ in s:
        count += 1
    return count

print(strlen('')) # 0
print(strlen('Python')) # 6
print(strlen('hello world')) # 11
```

Q89

CODING PROBLEM

Sum of digits of a number.

ANSWER

Convert the absolute value to a string, iterate over each character digit, convert back to int, and sum.

EXAMPLE

```
def digit_sum(n):
    return sum(int(d) for d in str(abs(n)))

print(digit_sum(0)) # 0
print(digit_sum(1234)) # 10
print(digit_sum(-9876)) # 30
```

Q90

CODING PROBLEM

Check if a number is an Armstrong (narcissistic) number.

ANSWER

An Armstrong number of d digits equals the sum of each digit raised to the power d. E.g. $153 = 1^3 + 5^3 + 3^3$.

EXAMPLE

```
def is_armstrong(n):
    digits = str(n)
    power = len(digits)
    return n == sum(int(d)**power for d in digits)

print(is_armstrong(1))      # True
print(is_armstrong(153))    # True (1+125+27=153)
print(is_armstrong(9474))   # True
print(is_armstrong(100))    # False
```

Q91

CODING PROBLEM

Generate Pascal's triangle.

ANSWER

Each row starts and ends with 1. Every interior value is the sum of the two values directly above it in the previous row.

EXAMPLE

```
def pascals(n):
    tri = []
    for i in range(n):
        row = [1] * (i + 1)
        for j in range(1, i):
            row[j] = tri[i-1][j-1] + tri[i-1][j]
        tri.append(row)
    return tri

for row in pascals(5):
    print(row)
# [1], [1, 1], [1, 2, 1], [1, 3, 3, 1], [1, 4, 6, 4, 1]
```

Q92

CODING PROBLEM

Find the longest substring without repeating characters.

ANSWER

Sliding window with a set tracking the characters in the current window. Shrink from the left when a duplicate is found.

EXAMPLE

```
def length_of_longest(s):
    seen = set()
    left = result = 0
    for right, ch in enumerate(s):
        while ch in seen:
            seen.remove(s[left]); left += 1
        seen.add(ch)
        result = max(result, right - left + 1)
    return result

print(length_of_longest('abcabcbb')) # 3
print(length_of_longest('pwwkew')) # 3
```

Q93

CODING PROBLEM

Implement an LRU cache from scratch.

ANSWER

OrderedDict maintains insertion order. Move a key to the end on access (most recently used). Evict from the front (least recently used) when capacity is exceeded.

EXAMPLE

```
from collections import OrderedDict

class LRUCache:
    def __init__(self, cap): self.cap=cap; self.c=OrderedDict()
    def get(self, k):
        if k not in self.c: return -1
        self.c.move_to_end(k); return self.c[k]
    def put(self, k, v):
        if k in self.c: self.c.move_to_end(k)
        self.c[k] = v
        if len(self.c) > self.cap: self.c.popitem(last=False)
```

Q94

CODING PROBLEM

Write a decorator that measures execution time.

ANSWER

Save `time.perf_counter()` before and after calling the wrapped function and print the delta. Use `functools.wraps` to preserve the original function's metadata.

EXAMPLE

```
import time
from functools import wraps

def timer(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        t0 = time.perf_counter()
        result = func(*args, **kwargs)
        print(f'{func.__name__}: {time.perf_counter()-t0:.4f}s')
        return result
    return wrapper

@timer
def slow(): return sum(range(5_000_000))
```

Q95

CODING PROBLEM

Implement the Singleton design pattern.

ANSWER

Override `__new__` to return the single shared instance. Subsequent instantiations return the same object rather than creating a new one.

EXAMPLE

```
class Singleton:
    _instance = None

    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

a = Singleton()
b = Singleton()
print(a is b)  # True
```


Q96

CODING PROBLEM

Write a function to deep clone a nested dict.

ANSWER

Recursively copy each value. When a value is itself a dict, recurse; otherwise return it directly. `copy.deepcopy()` does this in one call.

EXAMPLE

```
def deep_clone(d):
    if not isinstance(d, dict): return d
    return {k: deep_clone(v) for k, v in d.items()}

orig = {'a': 1, 'b': {'c': 2, 'd': [3,4]}}
clone = deep_clone(orig)
clone['b']['c'] = 99
print(orig['b']['c'])    # 2 - unaffected
```

Q97

CODING PROBLEM

Find the first non-repeating character in a string.

ANSWER

Build a frequency count with `Counter`, then iterate the original string to find the first character with `count == 1`.

EXAMPLE

```
from collections import Counter

def first_unique(s):
    counts = Counter(s)
    for ch in s:
        if counts[ch] == 1:
            return ch
    return None

print(first_unique('aabcddeff')) # 'b'
print(first_unique('aabb'))      # None
```

Q98

CODING PROBLEM

Implement a basic trie (prefix tree).

ANSWER

Each TrieNode holds a dict of child nodes (one per character) and an is_end flag. Insert walks character by character, creating nodes as needed.

EXAMPLE

```
class TrieNode:
    def __init__(self): self.ch={}; self.end=False

class Trie:
    def __init__(self): self.root = TrieNode()
    def insert(self, word):
        node = self.root
        for c in word:
            node = node.ch.setdefault(c, TrieNode())
        node.end = True
    def search(self, word):
        node = self.root
        for c in word:
            if c not in node.ch: return False
            node = node.ch[c]
        return node.end
```

Q99

CODING PROBLEM

Flatten a deeply nested dictionary.

ANSWER

Walk every key recursively. When a value is a nested dict, recurse and prepend the current key (with a separator). Otherwise record the leaf key-value pair.

EXAMPLE

```
def flatten_dict(d, parent='', sep='.'):
    items = []
    for k, v in d.items():
        key = f'{parent}{sep}{k}' if parent else k
        if isinstance(v, dict):
            items.extend(flatten_dict(v, key, sep).items())
        else:
            items.append((key, v))
    return dict(items)

d = {'a':1, 'b':{'c':2, 'd':{'e':3}}}
print(flatten_dict(d))
# {'a':1, 'b.c':2, 'b.d.e':3}
```

Q100

CODING PROBLEM

Implement an iterator class that generates perfect squares.

ANSWER

Implement `__iter__` (return self) and `__next__` (compute `current**2`, advance, or raise `StopIteration`). This makes the class usable in for loops and with `next()`.

EXAMPLE

```
class Squares:
    def __init__(self, limit):
        self.n = 1; self.limit = limit
    def __iter__(self): return self
    def __next__(self):
        if self.n > self.limit: raise StopIteration
        val = self.n ** 2; self.n += 1; return val

print(list(Squares(6)))
# [1, 4, 9, 16, 25, 36]
```